Phase #4 - Back End Rapid Prototype (CSCI 3060U)

Date: *March 16th 2026*
Group Members:

- Aliseena Ahmar (100870078)
- Haider Saleem (100870637)
- Samir Ahmadi (100916280)
- Smit Kalathia (100871659)

1. Introduction

The purpose of the Banking Back End is to read the old master accounts file and the merged transaction file, process each banking transaction in order, apply business rules and transaction fees, report errors to the terminal, and generate the updated master accounts file and current accounts file for the next day.

2. Class Intention Table

Class / Module	Type	Intention
Account	Class	Represents a single bank account in memory and stores the account data required during transaction processing.
Transaction	Class	Represents a transaction record read from the transaction file and stores its parsed data.
main.py	Module	Entry point of the Back End program which coordinates reading input files, processing transactions, and writing the final output file.
read.py	Module	Reads the current accounts file, validates each account record, and converts valid lines into Account objects.
transaction_reader.py	Module	Reads the transaction file, validates transaction records, and converts valid lines into Transaction objects.
backend_processor.py	Module	Processes transactions by applying each transaction to the appropriate account or accounts and updating the data in memory.

write.py	Module	Writes the updated account data stored in memory back to the output file using the required format.
print_error.py	Module	Handles formatted error reporting for both recoverable errors and fatal program errors.

3. Method / Function Tables

3.1 Account Class

Method	Intention
deposit(amount)	Adds the specified amount to the account balance and increments the transaction count.
withdraw(amount)	Subtracts the specified amount from the account balance and increments the transaction count.
transfer_out(amount)	Removes funds from the account when it is the source account in a transfer transaction.
transfer_in(amount)	Adds funds to the account when it is the destination account in a transfer transaction.
enable()	Sets the account status to active.
disable()	Sets the account status to disabled.
change_plan(new_plan)	Updates the account's plan code and increments the transaction count.
to_dict()	Converts the account object into a dictionary representation of its data for debugging or inspection.

3.2 Transaction Class

Method	Intention
from_line(line)	Parses a line from the transaction file and creates the corresponding Transaction object.
is_end_of_session()	Determines whether the transaction represents the end-of-session marker.

3.3 read.py

Function	Intention
read_current_accounts(filename)	Reads the current accounts file and returns a dictionary of parsed Account objects stored in memory.
parse_account_line(line, line_number)	Parses one fixed-width account record, validates the data, and converts it into an Account object.

3.4 transaction_reader.py

Function	Intention
read_transactions(filename)	Reads the transaction file and returns a list of parsed Transaction objects.
parse_transaction_line(line, line_number)	Parses and validates a transaction record and converts it into a Transaction object.

3.5 backend_processor.py

Function	Intention
process_transactions(accounts, transactions)	Processes each transaction sequentially and calls the appropriate handler function based on the transaction type.

handle_deposit(accounts, transaction)	Applies a deposit transaction to an existing account.
handle_withdrawal(accounts, transaction)	Applies a withdrawal transaction to an existing account.
handle_transfer(accounts, transaction)	Transfers money from one account to another based on the transaction information.
handle_create(accounts, transaction)	Creates a new account and adds it to the account dictionary if it does not already exist.
handle_delete(accounts, transaction)	Deletes an existing account from the accounts dictionary.
handle_disable(accounts, transaction)	Disables an existing account.
handle_change_plan(accounts, transaction)	Changes the plan type of an existing account if the provided plan code is valid.

4. System Inputs and Outputs

Input Files

The system reads the following input files:

- **Current Accounts File** – contains all existing bank accounts and their account data.
- **Transaction File** – contains all transactions to be processed by the back end system.

These files are stored in the **input directory**.

Output File

The system generates the following output file:

- **Updated Current Accounts File** – contains the updated account data after all transactions have been processed.

This file is written to the **output directory**.

5. UML Class Diagram